

《AI大模型Ragent项目》——知识库数据分块管理接口

来自： 拿个offer-开源&项目实战



2026年04月02日 22:12

上一篇讲完了文档的生命周期管理——删除、更新、启用/禁用，都是针对文档整体的操作。但文档粒度太粗了，自动分块之后，实际参与检索的是 Chunk。用户有时候需要针对某一段具体内容做调整，文档级操作够不到这个层面。自动分块用一套固定参数处理整份文档，大多数内容效果还行，但总有些段落切出来不对劲。常见的就两类问题：

- 一是**截断**，切割位置踩到一段完整描述的中间，前半截在一个 Chunk 里，后半截在下一个，各自语义残缺
- 二是**稀释**，某条关键但简短的条款被合并进前后的大段文字里，检索时语义被淹没，很难精准命中。这两种情况靠调参数解决不了——参数是全局的，顾了大段顾不了短句，顾了整体顾不了边角

最直接的方式是针对有问题的 Chunk 做手动调整：把截断的两段合并编辑成完整的 Chunk，把混入噪音的段落拆出来单独处理，或者把质量明显差的 Chunk 直接禁用。这比重新调整分块策略、重跑整个文档的代价小得多。更重要的是，即便调整策略重跑，边界情况永远存在——手动干预是兜底能力，不是临时替代方案。

这就是 Chunk 管理接口存在的原因：不是让用户从头手写知识库，而是给自动分块的结果提供精准干预的入口。

KnowledgeChunkController 共有 6 个接口，覆盖了 Chunk 的完整 CRUD 加启用/禁用控制。

六个接口概览

先整体看一下这 6 个接口的关键维度：

接口	HTTP 方法	路径	向量操作	事务方式
分页查询	GET	/knowledge-base/docs/{doc-id}/chunks	无	无事务
新增 Chunk	POST	/knowledge-base/docs/{doc-id}/chunks	写入向量库	@Transactional
更新内容	PUT	/knowledge-base/docs/{doc-id}/chunks/{chunk-id}	删旧插新	@Transactional
删除 Chunk	DELETE	/knowledge-base/docs/{doc-id}/chunks/{chunk-id}	精准删除单条	@Transactional
启用/禁用单条	PATCH	/knowledge-base/docs/{doc-id}/chunks/{chunk-id}/enable	按需写入或删除	@Transactional
批量启用/禁用	PATCH	/knowledge-base/docs/{doc-id}/chunks/batch-enable	精准写入或精准删除	编程式事务

pageQuery 接口详解

1. 接口与实现

```
@GetMapping("/knowledge-base/docs/{doc-id}/chunks")
public Result<IPage<KnowledgeChunkVO>> pageQuery(@PathVariable("doc-id") String docId,
                                                    @Validated KnowledgeChunkPageRequest requestParam) {
    return Results.success(knowledgeChunkService.pageQuery(docId, requestParam));
}
```

KnowledgeChunkPageRequest 继承了 MyBatis Plus 的 Page (带 current、size), 扩展了一个 enabled 字段用于状态过滤。

Service 层实现:

```
@Override
public IPage<KnowledgeChunkVO> pageQuery(String docId, KnowledgeChunkPageRequest requestParam) {
    KnowledgeDocumentDO documentDO = documentMapper.selectById(docId);
    Assert.notNull(documentDO, () -> new ClientException("文档不存在"));

    LambdaQueryWrapper<KnowledgeChunkDO> queryWrapper = new LambdaQueryWrapper<KnowledgeChunkDO>()
        .eq(KnowledgeChunkDO::getDocId, docId)
        .eq(requestParam.getEnabled() != null, KnowledgeChunkDO::getEnabled, requestParam.getEnabled())
        .orderByAsc(KnowledgeChunkDO::getChunkIndex);

    Page<KnowledgeChunkDO> page = new Page<>(requestParam.getCurrent(), requestParam.getSize());
    IPage<KnowledgeChunkDO> result = chunkMapper.selectPage(page, queryWrapper);
    return result.convert(each -> BeanUtil.toBean(each, KnowledgeChunkVO.class));
}
```

2. 设计要点

2.1 按 chunkIndex 排序

chunkIndex 记录的是 Chunk 在原文档中的顺序 (从 0 开始)。分块处理时按文档内容顺序生成, 用户在管理界面看 Chunk 列表时, 需要按这个顺序展示, 才能直观判断某段内容是文档的哪个部分、前后文是什么。

如果按 id 或 createTime 排序, 手动新增的 Chunk 会插到末尾, 即使它在逻辑上应该排在中间, 视觉上会很混乱。

2.2 enabled 过滤的实际用途

enabled 传 null 时查全部, 传 1 查已启用, 传 0 查已禁用。

这个过滤在实际使用中有几个场景: 看某文档里有哪些 Chunk 当前参与检索 (传 1); 审查被禁用的内容、确认是否需要恢复 (传 0); 查看全貌时不传, 看完整的分块结构。

create 接口详解

1. 接口定义

```
@PostMapping("/knowledge-base/docs/{doc-id}/chunks")
public Result<KnowledgeChunkVO> create(@PathVariable("doc-id") String docId,
    @RequestBody KnowledgeChunkCreateRequest request) {
    return Results.success(knowledgeChunkService.create(docId, request));
}
```

KnowledgeChunkCreateRequest 有三个字段: content (必填)、index (可选, 指定 Chunk 序号, 不传则自动追加到末尾)、chunkId (前端不会传, 文档操作接口里会用到)。

2. 核心实现流程

```
@Override
@Transactional(rollbackFor = Exception.class)
public KnowledgeChunkVO create(String docId, KnowledgeChunkCreateRequest requestParam) {
    KnowledgeDocumentDO documentDO = documentMapper.selectById(docId);
    Assert.notNull(documentDO, () -> new ClientException("文档不存在"));
    if (DocumentStatus.RUNNING.getCode().equals(documentDO.getStatus())) {
        throw new ClientException("文档正在分块处理中, 暂不支持新增 Chunk");
    }
    if (!Integer.valueOf(1).equals(documentDO.getEnabled())) {
        throw new ClientException("文档未启用, 暂不支持新增 Chunk");
    }

    String content = requestParam.getContent();
```

```

Assert.notBlank(content, () -> new ClientException("Chunk 内容不能为空"));

// 查当前最大 chunkIndex, 用于自动追加
KnowledgeChunkDO latest = chunkMapper.selectOne(
    Wrappers.lambdaQuery(KnowledgeChunkDO.class)
        .eq(KnowledgeChunkDO::getDocId, docId)
        .orderByDesc(KnowledgeChunkDO::getChunkIndex)
        .last("LIMIT 1")
);

// 优先使用请求中指定的 index, 未指定则自动追加到末尾
int chunkIndex = requestParam.getIndex() != null
    ? requestParam.getIndex()
    : (latest != null ? latest.getChunkIndex() + 1 : 0);

String contentHash = SecureUtil.sha256(content);
int charCount = content.length();
KnowledgeBaseDO kbDO = knowledgeBaseMapper.selectById(documentDO.getKbId());
String embeddingModel = kbDO.getEmbeddingModel();
String collectionName = kbDO.getCollectionName();
Integer tokenCount = resolveTokenCount(content);

KnowledgeChunkDO chunkDO = KnowledgeChunkDO.builder()
    .id(requestParam.getChunkId())
    .kbId(documentDO.getKbId())
    .docId(docId)
    .chunkIndex(chunkIndex)
    .content(content)
    .contentHash(contentHash)
    .charCount(charCount)
    .tokenCount(tokenCount)
    .enabled(1)
    .createdBy(UserContext.getUsername())
    .updatedBy(UserContext.getUsername())
    .build();

chunkMapper.insert(chunkDO);

// chunk_count 自增
documentMapper.update(Wrappers.lambdaUpdate(KnowledgeDocumentDO.class)
    .eq(KnowledgeDocumentDO::getId, docId)
    .setSql("chunk_count = chunk_count + 1"));

// 同步写入向量库
syncChunkToVector(collectionName, docId, chunkDO, embeddingModel);

return BeanUtil.toBean(chunkDO, KnowledgeChunkVO.class);
}

```

2.1 为什么要求文档启用才能新增 Chunk

前置校验有三个：文档存在 → status != RUNNING → enabled = 1。

前两个好理解，第三个值得说一下。禁用文档的语义是这份文档暂时下线，不参与检索。如果允许在禁用文档上新增 Chunk，新增的 Chunk 会立即写入向量库（create 方法末尾调用 syncChunkToVector），这个 Chunk 就进了向量库参与检索，但它的父文档已经被标记为禁用。数据库说不参与检索，向量库说可以被检索到，这就矛盾了。

更直接的说：禁用文档往往是因为这份文档的内容有问题，需要整体下线处理，这种时候往里面加内容没有意义。

2.2 chunkIndex：手动指定或自动追加

index 是可选参数。前端新建分块时可以指定序号，序号可以留空让系统自动追加到末尾。

```
// 优先使用请求中指定的 index，未指定则自动追加到末尾
int chunkIndex = requestParam.getIndex() != null
    ? requestParam.getIndex()
    : (latest != null ? latest.getChunkIndex() + 1 : 0);
```

自动追加时，查当前 docId 下 chunkIndex 最大值，加 1 作为新 Chunk 的序号。这里用查最大值 +1 而不是 COUNT(*)，是因为 Chunk 可能被删除，COUNT 的结果会低于最大 chunkIndex，追加进来的新 Chunk 序号就重复了。

比如文档原有 10 个 Chunk (0~9)，删掉了 Chunk 5 和 Chunk 8，COUNT 返回 8，新增 Chunk 的 chunkIndex 就成了 8，和原来的 Chunk 8 冲突（虽然原来的已被逻辑删除，但从业务语义上序号还是被占用了，容易引起混乱）。

手动指定序号的场景：用户发现自动分块在某个位置漏了一段内容，想在 Chunk 3 和 Chunk 4 之间插入一个新的，可以直接指定 index 为 3（或任意值）。注意这里不会自动调整其他 Chunk 的序号，因为 chunkIndex 只是排序用的显示字段，不是唯一约束，允许重复值。分页查询按 chunkIndex 升序排列，多个相同序号的 Chunk 会相邻显示。

2.3 contentHash 的作用

哈希计算直接用 Hutool 的工具方法，一行搞定：

```
String contentHash = SecureUtil.sha256(content);
```

为什么要存一个 contentHash？它的核心用途是**定时拉取场景下的增量更新判断**。

项目支持 URL 类型的文档定时拉取（ScheduleRefreshProcessor），每次定时任务触发时需要判断远程文档内容是否发生了变化。拉取到新内容后，计算 SHA-256 和上次存储的 contentHash 对比，如果一样就跳过重新分块和向量化，避免每次定时触发都做一遍完整的分块 + 向量化流程。

在 Chunk 级别，contentHash 也为将来的增量分块预留了能力：重跑分块时，对比新旧 Chunk 的 hash，内容没变的 Chunk 可以跳过向量化，只处理有变更的部分。当前代码里 update 接口的幂等检查用的是 newContent.equals(chunkDO.getContent()) 直接比字符串，还没有走 hash 对比，但 hash 存着不亏——字符串比较在内容很长时性能不如 hash 对比（64 字符定长 vs 可能几千字符的 content）。

2.4 tokenCount：估算值，仅供展示

```
Integer tokenCount = resolveTokenCount(content);
```

这里的 tokenCount 是一个**估算值**，不是精确的 Token 数。实际实现是 HeuristicTokenCounterService，用的是启发式规则：中文字符按 1 字 ≈ 1 Token，英文按 4 字符 ≈ 1 Token，其他字符按 2 字符 ≈ 1 Token。没有调用 Tokenizer 做精确分词，因为不同模型的 Tokenizer 不同，精确计算意义不大。

这个字段目前的用途是**管理后台的展示**——让知识库管理员在 Chunk 列表里直观看到每个分块的大致 Token 规模，方便判断哪些 Chunk 太长或太短。它不参与 RAG 检索链路的任何逻辑计算。

2.5 写库与向量同步的顺序

数据库写入成功后，调用 syncChunkToVector：

```
private void syncChunkToVector(String collectionName, String docId, KnowledgeChunkDO chunkDO, String content) {
    List<Float> embedding = embedContent(chunkDO.getContent(), embeddingModel);
    float[] vector = toArray(embedding);

    VectorChunk chunk = VectorChunk.builder()
        .index(chunkDO.getChunkIndex())
        .content(chunkDO.getContent())
        .chunkId(String.valueOf(chunkDO.getId()))
        .embedding(vector)
        .build();
    vectorStoreService.indexDocumentChunks(collectionName, docId, List.of(chunk));
}
```

整个 create 方法加了 @Transactional，但 syncChunkToVector（调用 Embedding API + 写向量库）是在事务内执行的。这里和 batchEnable 的做法不同：单条 Chunk 的向量化耗时短（一次 API 调用几百毫秒），放在事务内还可以接受；批量操作时 embed 多条，总耗时可能很长，那时候才需要把 embed 移到事务外。

update 接口详解

1. 接口定义

```
@PutMapping("/knowledge-base/docs/{doc-id}/chunks/{chunk-id}")
public Result<Void> update(@PathVariable("doc-id") String docId,
                           @PathVariable("chunk-id") String chunkId,
                           @RequestBody KnowledgeChunkUpdateRequest request) {
    knowledgeChunkService.update(docId, chunkId, request);
    return Results.success();
}
```

KnowledgeChunkUpdateRequest 只有一个字段：content。Chunk 的更新就是改内容，其他元数据（charCount、tokenCount、contentHash）由后端重新计算。

2. 核心实现流程

```
@Override
@Transactional(rollbackFor = Exception.class)
public void update(String docId, String chunkId, KnowledgeChunkUpdateRequest requestParam) {
    KnowledgeDocumentDO documentDO = documentMapper.selectById(docId);
    Assert.notNull(documentDO, () -> new ClientException("文档不存在"));
    if (DocumentStatus.RUNNING.getCode().equals(documentDO.getStatus())) {
        throw new ClientException("文档正在分块处理中，暂不支持修改 Chunk");
    }

    KnowledgeChunkDO chunkDO = chunkMapper.selectById(chunkId);
    Assert.notNull(chunkDO, () -> new ClientException("Chunk 不存在"));
    Assert.isTrue(chunkDO.getDocId().equals(docId), () -> new ClientException("Chunk 不属于该文档"));

    String newContent = requestParam.getContent();
    Assert.notBlank(newContent, () -> new ClientException("Chunk 内容不能为空"));

    // 内容没变，直接返回
    if (newContent.equals(chunkDO.getContent())) {
        return;
    }

    chunkDO.setContent(newContent);
    chunkDO.setContentHash(calculateHash(newContent));
    chunkDO.setCharCount(newContent.length());
    KnowledgeBaseDO kbDO = knowledgeBaseMapper.selectById(documentDO.getKbId());
    String embeddingModel = kbDO.getEmbeddingModel();
    String collectionName = kbDO.getCollectionName();
    chunkDO.setTokenCount(resolveTokenCount(newContent));
    chunkDO.setUpdatedBy(UserContext.getUsername());

    chunkMapper.updateById(chunkDO);

    // 同步更新向量库
    vectorStoreService.updateChunk(
        collectionName,
        docId,
        VectorChunk.builder()
            .chunkId(chunkId)
            .content(newContent)
            .index(chunkDO.getChunkIndex())
            .embedding(toArray(embedContent(newContent, embeddingModel)))
            .build()
    );
}
```

2.1 内容幂等检查

```
if (newContent.equals(chunkD0.getContent())) {  
    return;  
}
```

如果提交的内容和数据库里完全一样，直接返回，不触发向量更新。

这个检查是有实际价值的：Embedding API 是按 Token 计费的，一次无效的向量更新就是白白花钱。更重要的是，用户可能只是来确认内容的，打开 Chunk 编辑框看了一眼，直接点了保存，内容实际没有改动。算是一个优化项。

2.2 向量更新：为什么是删旧插新

vectorStoreService.updateChunk 这个方法名叫 update，但向量数据库底层的实现通常是删旧插新。向量数据库中存的是 (id, vector, payload)，虽然 chunkId 没变，但对应的 vector 完全不同了（内容变了，向量就变了），更新语义是替换向量数据，而不是修改某个值。

之前文章讲结果类似概念，就不再赘述了。

delete 接口详解

1. 接口定义

```
@DeleteMapping("/knowledge-base/docs/{doc-id}/chunks/{chunk-id}")  
public Result<Void> delete(@PathVariable("doc-id") String docId,  
                           @PathVariable("chunk-id") String chunkId) {  
    knowledgeChunkService.delete(docId, chunkId);  
    return Results.success();  
}
```

2. 核心实现流程

```
@Override  
@Transactional(rollbackFor = Exception.class)  
public void delete(String docId, String chunkId) {  
    KnowledgeDocumentD0 documentD0 = documentMapper.selectById(docId);  
    Assert.notNull(documentD0, () -> new ClientException("文档不存在"));  
    if (DocumentStatus.RUNNING.getCode().equals(documentD0.getStatus())) {  
        throw new ClientException("文档正在分块处理中，暂不支持删除 Chunk");  
    }  
  
    KnowledgeChunkD0 chunkD0 = chunkMapper.selectById(chunkId);  
    Assert.notNull(chunkD0, () -> new ClientException("Chunk 不存在"));  
    Assert.isTrue(chunkD0.getDocId().equals(docId), () -> new ClientException("Chunk 不属于该文档"));  
  
    KnowledgeBaseD0 kbD0 = knowledgeBaseMapper.selectById(documentD0.getKbId());  
    Assert.notNull(kbD0, () -> new ServiceException("知识库不存在"));  
    String collectionName = kbD0.getCollectionName();  
  
    chunkMapper.deleteById(chunkId);  
  
    // 安全递减 chunk_count  
    documentMapper.update(Wrappers.lambdaUpdate(KnowledgeDocumentD0.class)  
        .eq(KnowledgeDocumentD0::getId, docId)  
        .setSql("chunk_count = CASE WHEN chunk_count > 0 THEN chunk_count - 1 ELSE 0 END"));  
  
    deleteChunkFromVector(collectionName, chunkId);  
}
```

2.1 chunk_count 的 CASE WHEN 防护

```
chunk_count = CASE WHEN chunk_count > 0 THEN chunk_count - 1 ELSE 0 END
```

正常流程下，delete 方法已经校验了 Chunk 存在才走到这里，chunk_count 不应该是 0。但 chunk_count 是冗余计数字段，它的值不一定和实际 Chunk 行数严格一致——早期 bug、手动改库、并发事务回滚都可能导致偏差。加一层 CASE WHEN 保证不出负数，成本就是 SQL 多几个字符，属于低成本的防御性写法。

2.2 向量删除

```
private void deleteChunkFromVector(String collectionName, String chunkId) {  
    vectorStoreService.deleteChunkById(collectionName, chunkId);  
}
```

按 chunkId 删除对应的向量，只影响这一条，同文档其他 Chunk 的向量完全不受影响。和文档级的 deleteDocumentVectors（按 docId 删除该文档所有向量）不同，这里做的是精准的单条删除。

enable 接口详解

1. 接口定义

```
@PatchMapping("/knowledge-base/docs/{doc-id}/chunks/{chunk-id}/enable")  
public Result<Void> enable(@PathVariable("doc-id") String docId,  
    @PathVariable("chunk-id") String chunkId,  
    @RequestParam("value") boolean enabled) {  
    knowledgeChunkService.enableChunk(docId, chunkId, enabled);  
    return Results.success();  
}
```

2. 核心实现流程

```
@Override  
@Transactional(rollbackFor = Exception.class)  
public void enableChunk(String docId, String chunkId, boolean enabled) {  
    KnowledgeDocumentDO documentDO = documentMapper.selectById(docId);  
    Assert.notNull(documentDO, () -> new ClientException("文档不存在"));  
    if (DocumentStatus.RUNNING.getCode().equals(documentDO.getStatus())) {  
        throw new ClientException("文档正在分块处理中，暂不支持修改 Chunk 状态");  
    }  
    // 启用 Chunk 前校验文档本身是否启用  
    validateDocumentEnabledForChunkEnable(documentDO, enabled);  
  
    KnowledgeChunkDO chunkDO = chunkMapper.selectById(chunkId);  
    Assert.notNull(chunkDO, () -> new ClientException("Chunk 不存在"));  
    Assert.isTrue(chunkDO.getDocId().equals(docId), () -> new ClientException("Chunk 不属于该文档"))  
  
    // 幂等检查  
    int enabledValue = enabled ? 1 : 0;  
    if (chunkDO.getEnabled().equals(enabledValue)) {  
        return;  
    }  
  
    chunkDO.setEnabled(enabledValue);  
    chunkDO.setUpdatedBy(UserContext.getUsername());  
    chunkMapper.updateById(chunkDO);  
  
    KnowledgeBaseDO kbDO = knowledgeBaseMapper.selectById(documentDO.getKbId());  
    String collectionName = kbDO.getCollectionName();  
  
    if (enabled) {
```

```
String embeddingModel = kbD0.getEmbeddingModel();
syncChunkToVector(collectionName, docId, chunkD0, embeddingModel);
} else {
    deleteChunkFromVector(collectionName, chunkId);
}
}
```

2.1 启用 Chunk 前为什么要校验父文档是否启用

```
private void validateDocumentEnabledForChunkEnable(KnowledgeDocumentD0 documentD0, boolean enableC
    if (!enableChunk) {
        return;
    }
    if (!Integer.valueOf(1).equals(documentD0.getEnabled())) {
        throw new ClientException("文档未启用，无法启用Chunk，请先启用文档");
    }
}
```

禁用 Chunk 时不做这个校验（禁用操作本身不需要父文档处于任何特定状态）。启用 Chunk 时才检查。

原因和 create 接口一致：启用 Chunk 会把向量写入向量库，该 Chunk 就能被检索到。如果父文档是禁用状态，语义上这份文档不应该出现在检索结果里，但启用了某个 Chunk 之后它就出现了，逻辑矛盾。

实际使用中，用户如果想恢复一个被禁用文档里的某个 Chunk，系统会提示"请先启用文档"，引导他先把文档整体恢复，再决定哪些 Chunk 要保留/启用。

2.2 状态检查

```
int enabledValue = enabled ? 1 : 0;
if (chunkD0.getEnabled().equals(enabledValue)) {
    return;
}
```

当前状态已经是目标状态，直接返回。前端可能因为各种原因重复提交（用户快速连点、网络重试），加了这个检查，重复调用不产生副作用，也不会触发无意义的向量操作。

■ 同之前文档，需要配合防重复提交注解去做。毕竟如果有并发同时调用，获取的状态是一致的。

2.3 向量库按需同步

启用和禁用走两条不同的路径：

```
if (enabled) {
    // 重新向量化并写入向量库
    String embeddingModel = kbD0.getEmbeddingModel();
    syncChunkToVector(collectionName, docId, chunkD0, embeddingModel);
} else {
    // 从向量库删除
    deleteChunkFromVector(collectionName, chunkId);
}
```

启用：该 Chunk 的向量可能已被删除（之前被禁用时删掉了），需要重新 embed + 写入，让它重新参与检索。

禁用：从向量库删除该 Chunk 的向量，检索时就不会命中它。

batchEnable 接口详解

1. 接口定义


```
@PatchMapping("/knowledge-base/docs/{doc-id}/chunks/batch-enable")
public Result<Void> batchEnable(@PathVariable("doc-id") String docId,
                                @RequestParam("value") boolean enabled,
                                @RequestBody(required = false) KnowledgeChunkBatchRequest request) {
    knowledgeChunkService.batchToggleEnabled(docId, request, enabled);
    return Results.success();
}
```

KnowledgeChunkBatchRequest 就一个字段: List<String> chunkIds, 指定本次操作的 Chunk ID 列表。

2. 前置校验

```
@Override
public void batchToggleEnabled(String docId, KnowledgeChunkBatchRequest requestParam, boolean enabled) {
    if (requestParam == null || CollUtil.isEmpty(requestParam.getChunkIds())) {
        throw new ClientException("请指定需要操作的 Chunk, 全量启用/禁用请使用文档启用接口");
    }
    List<String> requestedIds = requestParam.getChunkIds();
    if (requestedIds.size() > 500) {
        throw new ClientException("单次批量操作 Chunk 数量不能超过 500");
    }

    KnowledgeDocumentDO documentDO = documentMapper.selectById(docId);
    Assert.notNull(documentDO, () -> new ClientException("文档不存在"));
    if (DocumentStatus.RUNNING.getCode().equals(documentDO.getStatus())) {
        throw new ClientException("文档正在分块处理中, 暂不支持批量修改 Chunk 状态");
    }
    validateDocumentEnabledForChunkEnable(documentDO, enabled);

    // 批量 ID 有效性校验
    List<KnowledgeChunkDO> found = chunkMapper.selectByIds(requestedIds);
    if (found.size() != requestedIds.size()) {
        throw new ClientException("存在无效的 Chunk ID, 请求 " + requestedIds.size() + " 个, 实际找到 " + found.size());
    }
    found.forEach(c -> {
        if (!c.getDocId().equals(docId)) {
            throw new ClientException("Chunk " + c.getId() + " 不属于文档 " + docId);
        }
    });
    List<String> targetIds = found.stream().map(KnowledgeChunkDO::getId).collect(Collectors.toList());

    // 过滤出需要变更的 Chunk
    int enabledValue = enabled ? 1 : 0;
    List<KnowledgeChunkDO> needUpdateChunks = chunkMapper.selectList(
        new LambdaQueryWrapper<KnowledgeChunkDO>()
            .in(KnowledgeChunkDO::getId, targetIds)
            .ne(KnowledgeChunkDO::getEnabled, enabledValue)
    );
    List<String> needUpdateIds = needUpdateChunks.stream().map(KnowledgeChunkDO::getId).collect(Collectors.toList());

    if (CollUtil.isEmpty(needUpdateIds)) {
        throw new ClientException(enabled ? "所有 Chunk 已全部启用, 无需重复操作" : "所有 Chunk 已全部禁用, 无需重复操作");
    }
    // ... 后续分两条路径
}
```

几个细节值得注意:

500 条上限：不是随意定的数字。一次批量 embed 500 条，调用 Embedding API 的延迟加上向量写入的耗时，总时间还在可接受范围内。超过这个量，接口响应时间可能就会比较长，用户体验变差，建议分批操作。

全部指定 ID 才处理：传进来的 chunkIds 如果有任何一个不存在或不属于该文档，整批操作都失败。这是为了保证调用方数据的准确性——如果前端传了一批 ID 里有几个已经被删了，与其静默忽略，不如直接报错，让调用方明确知道状态已经不一致了。

过滤已是目标状态的：从请求的 ID 列表中，只处理状态需要变更的那些 (ne(enabled, targetValue))。如果过滤完发现全部已是目标状态，直接抛异常提示用户，避免空跑一遍。

3. 两条路径的策略

启用和禁用走对称的策略：启用时只写入需要启用的向量，禁用时只删除需要禁用的向量。

3.1 批量启用：精准写入

```
if (enabled) {
    List<VectorChunk> vectorChunks = needUpdateChunks.stream()
        .map(c -> VectorChunk.builder()
            .chunkId(c.getId())
            .content(c.getContent())
            .index(c.getChunkIndex())
            .build())
        .collect(Collectors.toList());
    attachEmbeddings(vectorChunks, kbDO.getEmbeddingModel());

    transactionOperations.executeWithoutResult(status -> {
        chunkMapper.update(
            Wrappers.lambdaUpdate(KnowledgeChunkDO.class)
                .in(KnowledgeChunkDO::getId, needUpdateIds)
                .set(KnowledgeChunkDO::getEnabled, 1)
                .set(KnowledgeChunkDO::getUpdatedBy, UserContext.getUsername())
        );
        vectorStoreService.indexDocumentChunks(collectionName, docId, vectorChunks);
    });
}
```

只对本次需要从禁用变为启用的 Chunk 做向量化和写入，不动已有的向量。和单条启用 enableChunk 的策略一致——只处理变更的部分，Embedding API 调用量最小。

3.2 批量禁用：精准删除

```
} else {
    transactionOperations.executeWithoutResult(status -> {
        chunkMapper.update(
            Wrappers.lambdaUpdate(KnowledgeChunkDO.class)
                .in(KnowledgeChunkDO::getId, needUpdateIds)
                .set(KnowledgeChunkDO::getEnabled, 0)
                .set(KnowledgeChunkDO::getUpdatedBy, UserContext.getUsername())
        );
        vectorStoreService.deleteChunksByIds(collectionName, needUpdateIds);
    });
}
```

禁用方向逻辑一样：只删掉需要禁用的那批 Chunk 的向量，不影响其他的。

启用和禁用的策略是对称的：启用和禁用只变更请求的部分，都不动已有的向量。和单条操作 enableChunk 的思路一致——只处理需要变更的 Chunk，Embedding API 调用量最小。

4. 为什么用 TransactionOperations 而非 @Transactional

@Transactional 注解是方法级的，加了注解就意味着整个方法在一个事务里。batchToggleEnabled 里有 attachEmbeddings 调用，它是对 Embedding API 的批量请求，涉及网络 IO，耗时可能较长。把这个操作放在事务里，就意味着一个数据库连接在等待 Embedding API 响应期间一直被占用，连接池资源被拉住，并发处理能力下降。

编程式事务 transactionOperations.executeWithoutResult 允许精确控制事务范围：

```
// 事务外：耗时的 embed API 调用
attachEmbeddings(vectorChunks, kbD0.getEmbeddingModel());

// 事务内：只包含 DB 写入 + 向量库操作
transactionOperations.executeWithoutResult(status -> {
    chunkMapper.update(...);
    vectorStoreService.indexDocumentChunks(...);
});
```

embed 完成之后，快速开启事务，做完 DB 和向量库操作，立刻提交。事务持有时间很短，数据库连接的占用时间也短。同样的模式在之前的 `persistChunksAndVectorsAtomically` 里也用了——分块处理完生成大批 Chunk 后，先算好所有向量，再开事务写库，事务内不做任何耗时操作。下面是一个直观对比：

方式	事务范围	连接占用时间	适用场景
@Transactional	整个方法	包含 embed 耗时（长）	单条操作，无外部 IO
TransactionOperations	手动指定代码块	只含 DB 写入（短）	含外部 IO 的批量操作

还有一个细节：`batchToggleEnabled` 方法本身没有加 `@Transactional`，这是对的。如果外层方法加了注解，内部的 `transactionOperations.executeWithoutResult` 会加入外层事务而不是开新事务，反而失去了精确控制的效果。

事务与数据一致性小结

1. 各接口的事务对比

接口	事务方式	向量库操作位置
pageQuery	无事务	无向量操作
create	@Transactional	事务内
update	@Transactional	事务内
delete	@Transactional	事务内
enableChunk	@Transactional	事务内
batchToggleEnabled	编程式事务（TransactionOperations）	embed 在事务外；DB + 向量库操作在事务内

单条操作的 Embedding API 调用耗时在几百毫秒，放事务内可以接受，写法简单。批量操作的 embed 耗时随数量线性增长，必须移到事务外。这个边界判断是实用主义的权衡，不是固定规范。

2. 向量库的一致性保障

DB 是主数据，向量库是派生索引，基本原则：

- 操作顺序：先成功写 DB，再操作向量库
- 事务回滚时：DB 操作回滚，向量库操作通常已经执行（非 Pgvector 向量库不支持 Spring 事务），存在短暂不一致
- 向量库写入失败时：当前接口会抛异常，DB 操作随事务回滚，数据库保持干净；向量库（非 Pgvector）里可能残留脏数据，但下次成功操作（比如重新执行分块）能修复

FAQ

1. 为什么新增 Chunk 要求文档是启用状态

新增 Chunk 会立即写入向量库参与检索。如果文档被标记为禁用，说明整体已下线，此时往向量库写内容，会让该文档的数据重新出现在检索结果里，与文档禁用的状态产生矛盾。

2. update 接口内容没变为什么要直接返回不触发向量更新

Embedding API 是按 Token 计费的，内容没变意味着向量也没变，触发更新是纯粹的浪费。而且向量更新底层是删旧插新，中间状态有短暂的窗口该 Chunk 不在向量库里，影响实时检索。内容没变直接跳过，不产生任何实际副作用。

3. updateChunk 底层为什么是删旧插新

向量是由内容生成的，内容变了，整个向量就变了，没有“修改部分字段”的说法。向量数据库的更新操作本质上就是删除旧向量 + 插入新向量，updateChunk 这个方法名只是在语义层面做了封装，底层实现就是 delete + insert。

4. 批量启用为什么不做全量重建

单条启用 enableChunk 只写入该条 Chunk 的向量，如果批量启用改成全量重建（清空所有向量再全部重写），两者逻辑就不一致——如果全量重建是必要的，单条启用也应该做；如果不是必要的，批量启用也没必要做。所以批量启用和单条启用保持一致：只对本次需要变更的 Chunk 做向量化写入，不动已有的向量，Embedding API 调用量最小。

5. 为什么 batchEnable 用 TransactionOperations 而不是 @Transactional

@Transactional 是方法级事务，attachEmbeddings（调用 Embedding API）会被包进事务里，导致数据库连接在等待 API 响应期间一直被占用。批量操作可能 embed 数十个 Chunk，这个时间不短。用编程式事务可以把耗时的外部调用放在事务外，只对 DB 写入和向量库操作开事务，连接占用时间大幅缩短。

6. 批量操作为什么限制 500 条

超过 500 条时，embed 的网络请求时间和向量库写入时间加起来会让接口响应变得很慢，用户体验明显下降。500 是一个经验值，在当前硬件和网络条件下，这个量级的操作完成时间是可接受的。实际业务中很少有一次性需要批量改动超过 500 个 Chunk 的情况；如果真有，分批调用接口即可。

7. Chunk 归属校验是防止什么

URL 路径里同时传了 doc-id 和 chunk-id，但 chunk-id 在数据库里是全局唯一的，不需要文档 ID 就能定位到一条记录。归属校验是防止越权操作：用文档 A 的路径，传文档 B 的 chunk-id，如果不校验归属关系，就能绕过文档权限对其他文档的 Chunk 做修改、删除、启用/禁用操作。在引入了权限控制（比如多租户场景下）的系统里，这个校验是必要的安全防线。